

Forecasting Burger Sales Using Machine Learning and Deep Learning Models

Author: Benard Manderera

Available at:

https://github.com/Bennyaks/Burger_Sales_ForecastingDate: July 2026

Technologies: Python, Pandas, NumPy, Scikit-Learn, LightGBM, TensorFlow/Keras, AWS SageMaker

Abstract

This project investigates the use of machine learning and deep learning techniques for forecasting daily burger sales across a ten-region restaurant chain using historical weather observations and engineered temporal features. The dataset comprises 24,500 daily records spanning January 2014 to September 2020, containing temperature, humidity, wind, visibility, atmospheric pressure, and sales figures for each region.

Following data cleaning, exploratory analysis, and feature engineering — which introduced calendar attributes, sales lag features, and rolling statistics — three predictive models were developed and compared: a LightGBM gradient boosting regressor, a baseline Long Short-Term Memory (LSTM) network, and an improved second LSTM architecture with revised scaling and a deeper topology.

The LightGBM model substantially outperformed both LSTM variants, achieving a test MAE of 353.58, RMSE of 478.57, R^2 of 0.8068, and SMAPE of 30.61%, compared with R^2 values below 0.02 for both LSTM experiments. These results demonstrate that for this structured, tabular time-series problem, gradient-boosted trees exploiting engineered lag and rolling features considerably outperform recurrent neural networks, despite the latter's theoretical advantage in modelling sequential dependencies. The report closes with a discussion of these findings, their business implications, and recommendations for further work including hyperparameter optimization with Optuna, additional tree-based baselines, and production deployment via AWS SageMaker.

Table of Contents

Abstract.....	2
Table of Contents.....	3
Chapter 1: Introduction	5
1.1 Background	5
1.2 Business Problem	5
1.3 Motivation.....	5
1.4 Project Objectives	5
1.5 Scope.....	5
Chapter 2: Dataset Description.....	6
2.1 Source	6
2.2 Variables.....	6
2.3 Dataset Statistics.....	6
2.4 Target Variable.....	6
2.5 Data Quality	6
Chapter 3: Exploratory Data Analysis	7
3.1 Sales Distribution	7
3.2 Correlation Heatmap	7
3.3 Regional Sales	8
3.4 Weather Relationships.....	9
Chapter 4: Data Cleaning	11
4.1 Missing Values.....	11
4.2 Pressure Anomalies.....	11
4.3 Duplicate Handling	11
4.4 Outlier Treatment	11
Chapter 5: Feature Engineering	12
5.1 Calendar Features	12
5.2 Lag Features	12
5.3 Rolling Means.....	13
5.4 Rolling Standard Deviations	13
5.5 Impact of Feature Engineering.....	13
Chapter 6: Data Preprocessing	14
6.1 Chronological Train-Validation-Test Split	14
6.2 Label Encoding	14

6.3 Quantile Transformation.....	14
6.4 MinMax Scaling.....	14
6.5 Sequence Generation.....	14
Chapter 7: LightGBM Development.....	16
7.1 Algorithm Overview	16
7.2 Hyperparameters	16
7.3 Training	16
7.4 Feature Importance	16
7.5 Performance	17
Chapter 8: LSTM Experiment 1	19
8.1 Architecture	19
8.2 Results.....	19
8.3 Weaknesses	19
Chapter 9: LSTM Experiment 2	21
9.1 Improved Preprocessing	21
9.2 Target Scaling.....	21
9.3 Improved Architecture.....	21
9.4 Results.....	21
Chapter 10: Model Comparison.....	22
Chapter 11: Discussion.....	23
11.1 Why LightGBM Dominated	23
11.2 Why LSTM Struggled	23
11.3 Weather and Calendar Variables Are Tabular Data.....	23
11.4 Deep Learning Isn't Always the Best Choice.....	23
Chapter 12: Conclusion	25
12.1 Objectives Achieved.....	25
12.2 Key Findings	25
12.3 Business Implications	25
Chapter 13: Lesson learnt	26
References	27

Chapter 1: Introduction

1.1 Background

Weather is widely recognized as a factor influencing consumer demand for food and beverage products. Retail food chains routinely observe that variables such as temperature, humidity, and precipitation-related conditions correlate with shifts in customer footfall and purchasing behaviour. For a burger restaurant chain operating across multiple regions, understanding and anticipating these fluctuations is central to effective inventory, staffing, and supply-chain decisions.

1.2 Business Problem

The restaurant chain operates retail stores across ten regions and wants to determine whether weather conditions measurably affect burger sales, and if so, to build a system capable of forecasting next-day sales using historical weather and sales data. Accurate forecasts allow the business to reduce food waste, optimize staffing levels, and improve supply-chain planning.

1.3 Motivation

Traditional forecasting approaches often rely on simple historical averages or naive extrapolation, which fail to account for external drivers such as weather. Machine learning and deep learning methods offer the potential to learn complex, non-linear relationships between weather, calendar effects, and sales, but the relative effectiveness of tree-based versus sequence-based (recurrent) approaches on this type of structured, tabular time-series data is not obvious in advance. This project directly compares both families of models to determine which is best suited to the problem.

1.4 Project Objectives

- Determine whether weather-related features influence burger sales.
- Engineer calendar-based and historical sales features to support forecasting.
- Build and tune a LightGBM baseline model.
- Build and evaluate LSTM deep learning models for the same forecasting task.
- Compare model performance using standard regression metrics.
- Prepare the best-performing model for deployment via AWS SageMaker.

1.5 Scope

The scope of this project covers data cleaning, exploratory data analysis, feature engineering, model development for both LightGBM and LSTM architectures, rigorous chronological evaluation, and preparation of deployment artifacts for AWS SageMaker. The project does not cover live production deployment, real-time data ingestion, or integration with point-of-sale systems, which are identified as future work.

Chapter 2: Dataset Description

2.1 Source

The dataset was provided as a CSV export representing daily observations originally sourced from a MySQL database. It contains 24,500 rows and 8 columns, covering the period from 1 January 2014 to 15 September 2020 across ten store regions (Reg1 through Reg10).

2.2 Variables

Feature	Description
Date	Observation date (DD-MM-YYYY)
Region	Restaurant region (Reg1 – Reg10)
Temperature	Daily average temperature
Humidity	Daily relative humidity (0–1 scale)
Wind	Wind speed
Visibility	Visibility distance
Pressure	Atmospheric pressure (hPa)
Sales	Daily burger sales (target variable)

2.3 Dataset Statistics

Prior to cleaning, the raw dataset contained 24,500 rows with no null values in any column. Each of the ten regions contributed between roughly 2,442 and 2,493 records. After removing duplicate rows and engineering additional features, the final modelling dataset contained 24,422 observations across 19 predictive features plus the target.

2.4 Target Variable

The target variable is Sales, representing daily burger sales revenue per region. Sales values in the raw data range from approximately 0.16 to 3,564, with a mean around 1,873 and considerable day-to-day variability, reflecting the volatility inherent in daily retail demand.

2.5 Data Quality

- No missing values were present in any of the eight original columns.
- 48 fully duplicated rows were identified, concentrated on specific dates within Region 1, and removed.
- 44 records contained an atmospheric pressure value of exactly 0 hPa — a physically implausible reading — which was treated as an anomaly and corrected.
- The Date column was stored as text and required conversion to a proper datetime type before any time-based analysis or feature engineering could proceed.

Chapter 3: Exploratory Data Analysis

Exploratory data analysis (EDA) was carried out to understand the distribution of burger sales, evaluate relationships between weather variables and sales, and surface any data quality issues requiring correction before modelling.

3.1 Sales Distribution

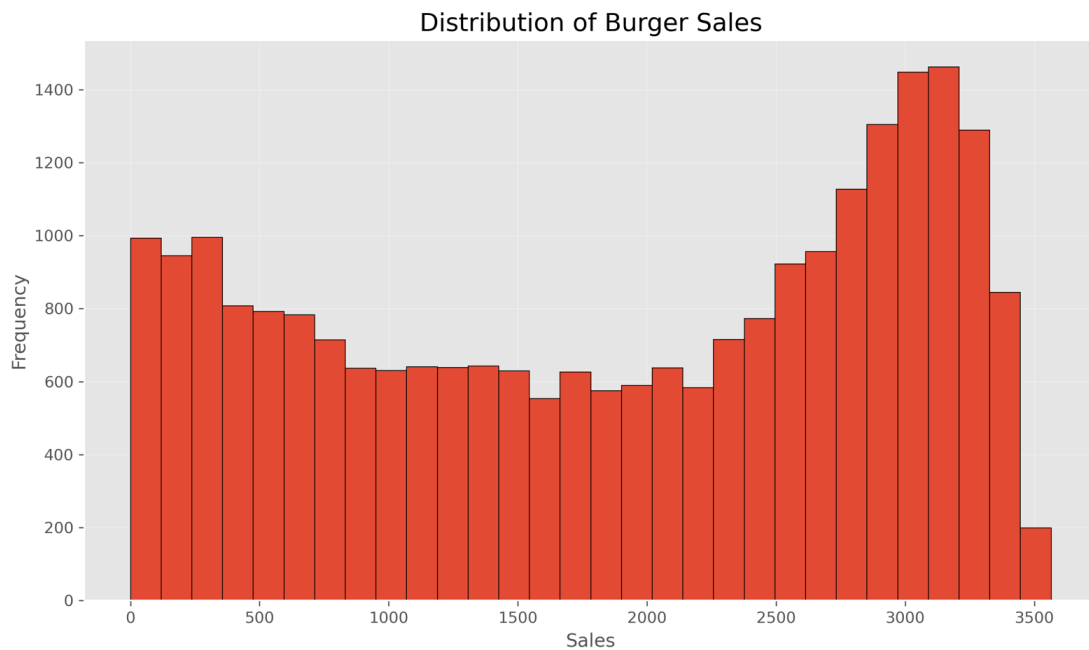


Figure 3.1 – Distribution of Burger Sales

The distribution of daily sales is broad and right-skewed, with values spanning from near zero to over 3,500. This spread confirms that sales are highly variable day to day, which motivates the use of models capable of capturing complex, non-linear relationships rather than simple averaging techniques.

3.2 Correlation Heatmap

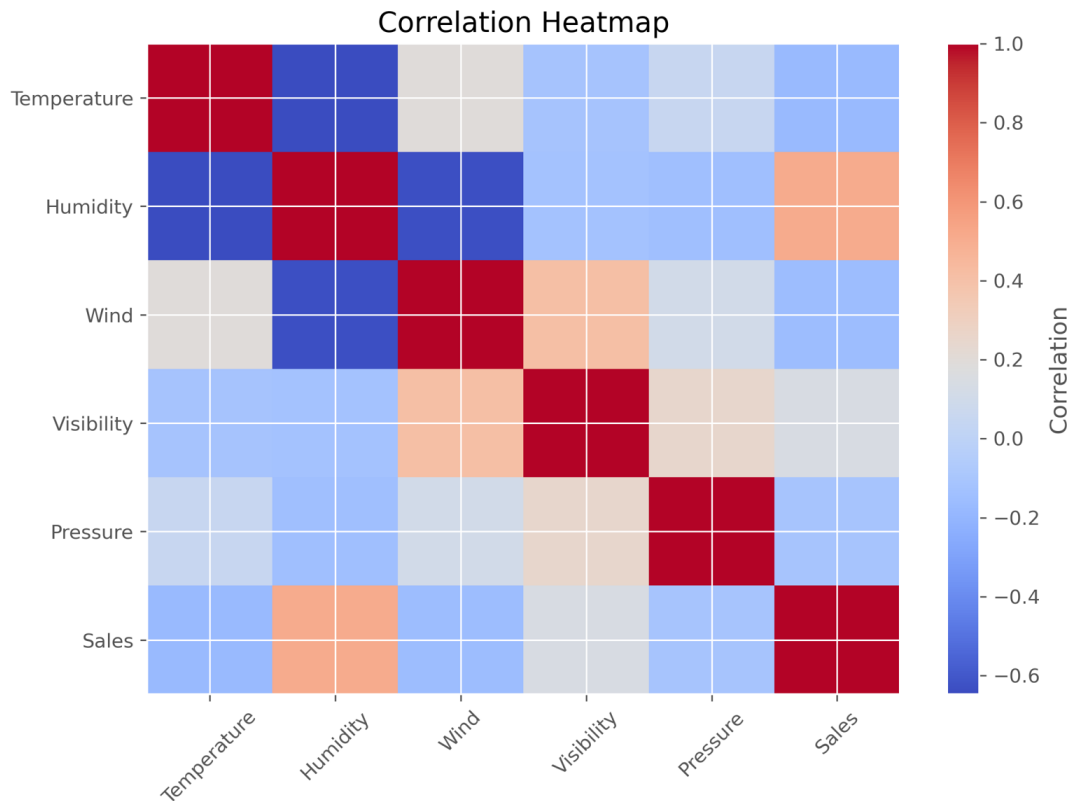


Figure 3.2 – Correlation Heatmap of Weather Variables and Sales

Correlation analysis was used as an initial exploratory tool to identify linear relationships between weather variables and sales. Humidity showed the strongest positive correlation with sales at 0.506, suggesting it may be an informative predictor. Temperature, wind, visibility, and pressure each displayed comparatively weak linear correlations with sales (all with absolute values below 0.18). Notably, temperature and humidity were moderately negatively correlated with one another (-0.647), consistent with the general climatological tendency for warmer days to coincide with lower relative humidity in this dataset. Because correlation only captures linear relationships, these results served primarily to guide feature engineering rather than to rule out any variable's predictive value — the LightGBM and LSTM models developed later are both capable of learning non-linear patterns the correlation matrix cannot detect.

3.3 Regional Sales

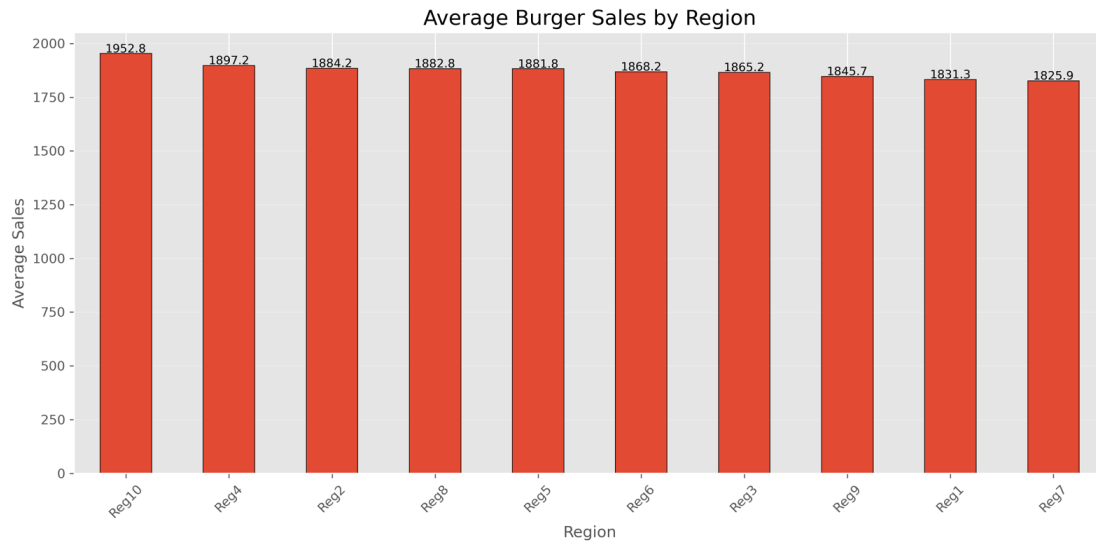


Figure 3.3 – Average Burger Sales by Region

Region 10 recorded the highest average sales (1,952.8), while Region 7 recorded the lowest (1,825.9) — a spread of roughly 127 units, or about 6.5% of the mean. This indicates that regional differences exist but are relatively modest; region alone does not explain the majority of sales variability. Region was nonetheless retained as a model feature, since regional weather patterns and local demand characteristics may still interact meaningfully with other predictors.

3.4 Weather Relationships

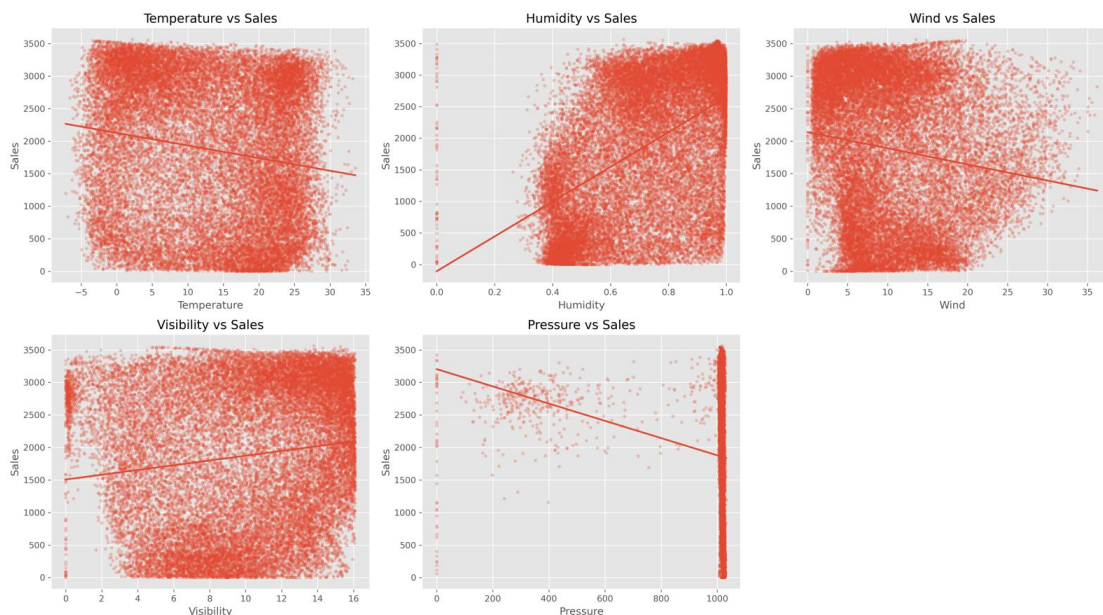


Figure 3.4 – Weather Variables versus Sales, with Linear Trend Lines

Scatter plots of each weather variable against sales, overlaid with linear trend lines, confirm the correlation analysis: humidity shows the clearest positive association with sales, while temperature and

wind show weak negative trends and visibility shows only a slight positive relationship. None of the relationships is strongly linear, reinforcing the earlier conclusion that non-linear or ensemble modelling approaches are appropriate for this problem. This analysis also surfaced an anomaly in the pressure variable — a cluster of readings fixed at exactly 0 hPa — which is addressed in the data cleaning stage described in Chapter 4.

Chapter 4: Data Cleaning

4.1 Missing Values

The raw dataset contained no missing values in any of its eight original columns, as confirmed by an explicit null check across the full 24,500-row dataset. However, feature engineering carried out later (Chapter 5) introduced missing values at the start of the time series, since lag and rolling-window features require a certain amount of preceding history to compute. These engineered-feature nulls (affecting a maximum of 30 leading rows) were removed prior to modelling, since they could not be meaningfully imputed without introducing look-ahead bias.

4.2 Pressure Anomalies

Inspection of the Pressure column revealed 44 records with a value of exactly 0.0 hPa — a physically implausible reading, since typical atmospheric pressure at sea level is approximately 1,013 hPa and no genuine observation would read zero. These values were treated as sensor or data-entry anomalies rather than genuine measurements. Rather than dropping these rows outright and losing their associated sales and other weather readings, each zero was replaced with the median pressure calculated from all valid (non-zero) observations, 1,016.49 hPa. Using the median rather than the mean protects against distortion from any remaining outliers in the valid readings.

4.3 Duplicate Handling

A duplicate check across all columns identified 48 fully duplicated rows. Investigation showed these duplicates were not randomly distributed: they were concentrated on a small number of specific dates within Region 1 — for example, 1 January 2014 appeared eleven times with identical values across every field, while most other dates for that region appeared only once. Because these rows were identical across every column (weather readings and sales alike), they were judged to be erroneous repeated entries rather than genuine independent observations, and were removed using `drop_duplicates()`, reducing the dataset from 24,500 to 24,452 rows.

4.4 Outlier Treatment

Beyond the pressure anomaly described above, the dataset was reviewed for other extreme or implausible values across the weather and sales columns. No further systematic anomalies were identified: the remaining variation in sales, despite its wide range, was judged to reflect genuine business volatility rather than data errors, and was therefore retained rather than clipped or removed. This decision preserves the natural variability that the forecasting models are intended to learn from.

Chapter 5: Feature Engineering

Weather variables alone may not fully explain burger sales. Customer purchasing behaviour is also shaped by calendar effects (day of week, seasonality) and by recent sales momentum. This chapter describes the additional features engineered to capture these effects.

Raw Data



Calendar Features



Lag Features



Rolling Features



Final Dataset

5.1 Calendar Features

- Year – the calendar year of the observation, allowing the model to capture any long-term trend across the 2014–2020 period.
- Month – the calendar month (1–12), capturing seasonal effects such as summer versus winter demand patterns.
- Day – the day of the month (1–31).
- Quarter – the calendar quarter (1–4), a coarser seasonal signal than month.
- Week Number – the ISO week of the year (1–53), providing a finer-grained seasonal signal than month or quarter.
- Weekend Flag (IsWeekend) – a binary indicator (1 if Saturday or Sunday, else 0), since restaurant demand often differs systematically between weekdays and weekends.

5.2 Lag Features

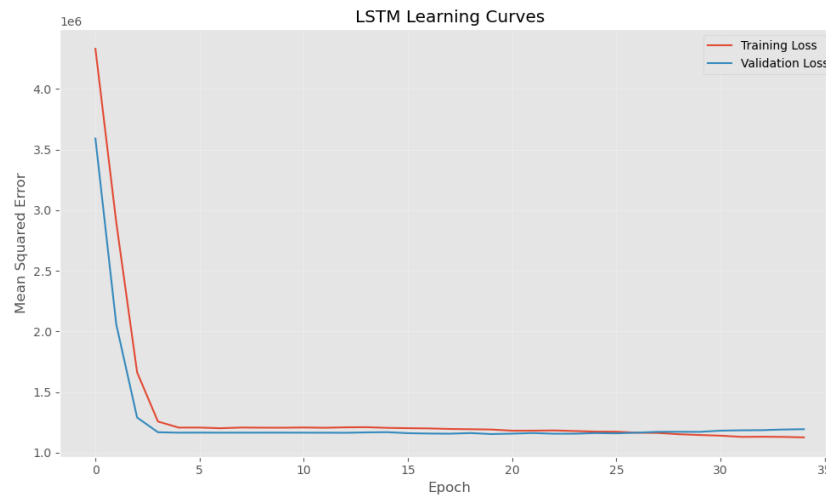
Lag features carry forward previous sales values as predictors of the current day's sales, on the premise that today's demand is often related to recent demand levels.

- Sales_Lag_1 – sales value from the immediately preceding record.
- Sales_Lag_7 – sales value from seven records (approximately one week) earlier, capturing weekly seasonality.
- Sales_Lag_30 – sales value from thirty records (approximately one month) earlier, capturing monthly patterns.

5.3 Rolling Means

Rolling mean features smooth out day-to-day noise and reveal underlying sales trends.

- Rolling_Mean_7 – the mean of the previous 7 sales values (computed using a value shifted by one period, so the current day’s own sales cannot leak into its own feature).
- Rolling_Mean_30 – the mean of the previous 30 sales values, capturing a longer-term trend.



5.4 Rolling Standard Deviations

- Rolling_STD_7 – the standard deviation of the previous 7 sales values, capturing recent sales volatility as a feature in its own right.

5.5 Impact of Feature Engineering

After introducing these calendar, lag, and rolling features, the leading rows of the dataset that lacked sufficient history to compute the 30-day rolling and lag windows (30 rows in total) were removed, leaving a final feature-engineered dataset of 24,422 rows across 21 columns (19 predictive features plus Date and Sales). This engineered feature set forms the input to all subsequent modelling in Chapters 6 through 10. Later analysis (Chapter 7) confirms that several of these engineered features — particularly Rolling_Mean_30, Sales_Lag_7, Sales_Lag_30, and Rolling_Mean_7 — rank among the most influential predictors in the LightGBM model, alongside the core weather variables.

Chapter 6: Data Preprocessing

6.1 Chronological Train-Validation-Test Split

Because this is a time-series forecasting problem, the dataset was split chronologically rather than randomly, to prevent future information from leaking into model training — a critical requirement for producing a realistic evaluation of forecasting performance. The engineered dataset (24,422 rows) was split 70% training, 15% validation, and 15% testing, in strict date order:

Split	Rows
Training	17,095
Validation	3,663
Testing	3,664

2014 ----- 2020

|-----70%-----|

|--15%--|

|--15%--|

Training Validation Test

6.2 Label Encoding

The categorical Region feature (Reg1 through Reg10) was converted into numeric form using scikit-learn's LabelEncoder, mapping each region string to an integer code (0–9). The fitted encoder was persisted with joblib so that identical encoding can be reproduced at inference time.

6.3 Quantile Transformation

For the LightGBM pipeline, all 19 input features were scaled using a QuantileTransformer configured with a normal output distribution, fitted exclusively on the training split and then applied to the validation and test splits. Quantile transformation maps each feature to a specified distribution based on its rank, which is robust to outliers and skewed feature distributions — a useful property given the varied scales and distributions of the weather and engineered features.

6.4 MinMax Scaling

For the improved LSTM pipeline (Experiment 2, Chapter 9), a different scaling strategy was used: a MinMaxScaler was applied to the input features, and a separate MinMaxScaler was fitted independently to the target variable (Sales). Neural networks are particularly sensitive to input scale, and scaling the target as well as the features is a standard technique to stabilize and speed up gradient-based optimization. Predictions were subsequently inverse-transformed back to the original sales scale for evaluation.

6.5 Sequence Generation

LSTM models require sequential input rather than independent rows. A sliding-window function was used to convert the scaled tabular data into overlapping sequences: each input sample consists of the preceding 30 days of observations across all 19 features, with the target being the sales value on the day immediately following that window. This produced sequence tensors of shape (samples, 30, 19) for training, validation, and test splits alike (for example, 17,065 training sequences after accounting for the 30-day lookback).

Chapter 7: LightGBM Development

7.1 Algorithm Overview

LightGBM (Light Gradient Boosting Machine) is a gradient boosting framework that builds an ensemble of decision trees sequentially, with each new tree trained to correct the residual errors of the previous ensemble. It is well suited to structured, tabular data and is known for fast training speed and strong out-of-the-box performance, making it a natural baseline model for this forecasting task.

7.2 Hyperparameters

An initial LightGBM model was trained with manually chosen hyperparameters (500 estimators, learning rate 0.05, max depth 8, 31 leaves, 0.8 subsample and column sampling ratios). This baseline was then improved using RandomizedSearchCV combined with TimeSeriesSplit (5 folds) to preserve chronological validity during cross-validation, searching over 20 candidate hyperparameter combinations. The best configuration identified was:

Hyperparameter	Value
n_estimators	700
learning_rate	0.01
max_depth	-1 (unlimited)
num_leaves	70
subsample	0.9
colsample_bytree	1.0
min_child_samples	20

7.3 Training

The tuned model was trained on the 17,095-row training split (quantile-scaled, 19 features), with the validation split used both during hyperparameter search and for monitoring generalization performance.

7.4 Feature Importance

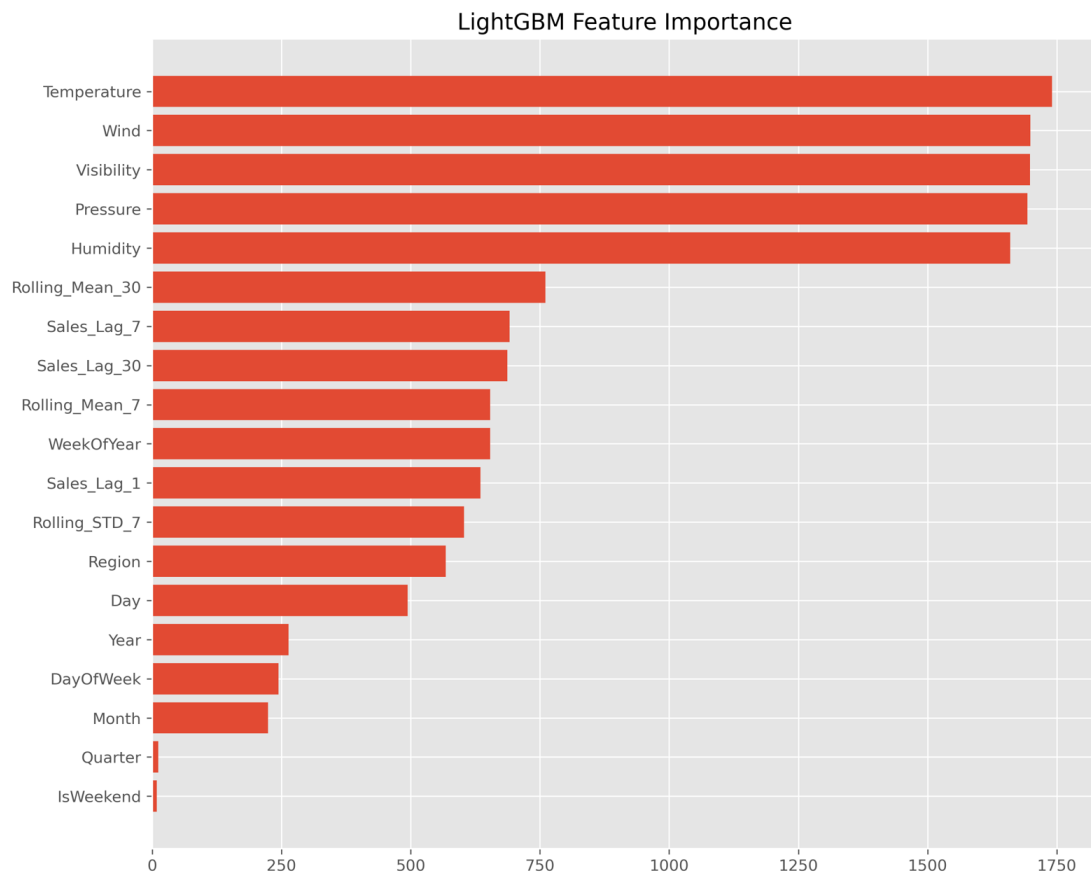


Figure 7.1 – LightGBM Feature Importance

The five raw weather variables — Temperature, Wind, Visibility, Pressure, and Humidity — emerged as the most important predictors by split count, closely followed by the engineered Rolling_Mean_30, Sales_Lag_7, Sales_Lag_30, and Rolling_Mean_7 features, along with WeekOfYear. This indicates that both the raw weather signal and the engineered historical-sales features contribute meaningfully to the model’s predictive power.

7.5 Performance

Dataset	MAE	RMSE	R ²	SMAPE
Training	282.69	376.99	0.8796	25.32%
Validation	340.10	465.64	0.8135	28.87%
Testing	345.50	468.60	0.8147	30.23%

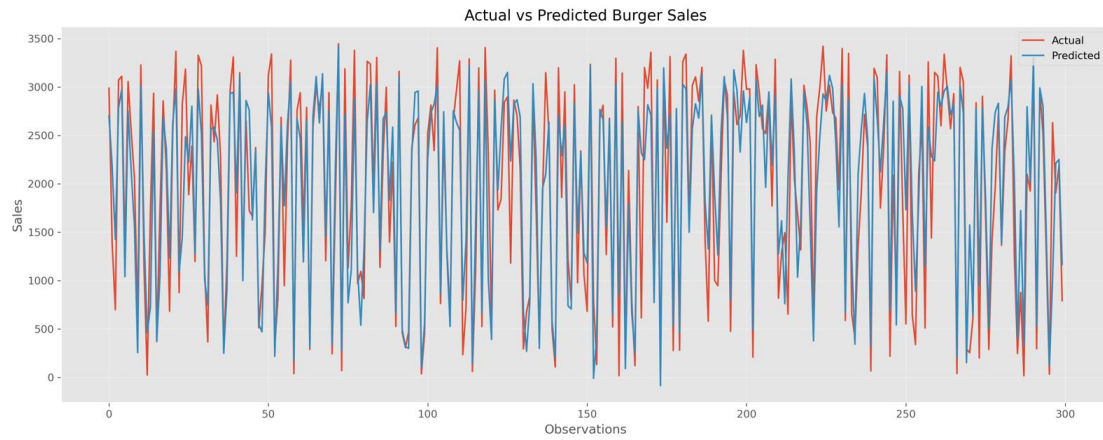


Figure 7.2 – LightGBM Predictions vs Actual Sales (Test Set, first 300 observations)

The tuned LightGBM model explained approximately 81% of the variance in test-set sales ($R^2 \approx 0.81$) with a SMAPE of roughly 30%, indicating strong and consistent forecasting performance across all three data splits with no evidence of severe overfitting (training and test R^2 remain close). This model was saved as the final LightGBM artifact and serves as the benchmark against which both LSTM experiments are compared in Chapter 10.

Chapter 8: LSTM Experiment 1

8.1 Architecture

The first LSTM experiment used a relatively compact recurrent architecture, reflecting the moderate size of the dataset and a preference for avoiding unnecessary overfitting risk:

- LSTM layer 1: 64 units, return_sequences=True, input shape (30 timesteps × 19 features)
- Dropout: 20%
- LSTM layer 2: 32 units
- Dropout: 20%
- Dense hidden layer: 16 units, ReLU activation
- Output layer: 1 unit (linear), predicting next-day sales directly in original units

The model was compiled with the Adam optimizer and mean squared error (MSE) loss, and trained for up to 100 epochs with EarlyStopping (patience 15, restoring best weights), ModelCheckpoint, and ReduceLROnPlateau callbacks. Critically, in this first experiment the target variable (Sales) was not separately scaled — the model was trained to predict raw sales values directly, using the same QuantileTransformer-scaled features as the LightGBM pipeline.

8.2 Results

Dataset	MAE	RMSE	R ²	SMAPE
Training	959.44	1074.59	0.0219	60.89%
Validation	955.30	1074.05	0.0098	60.86%
Testing	969.92	1088.14	0.0017	62.15%

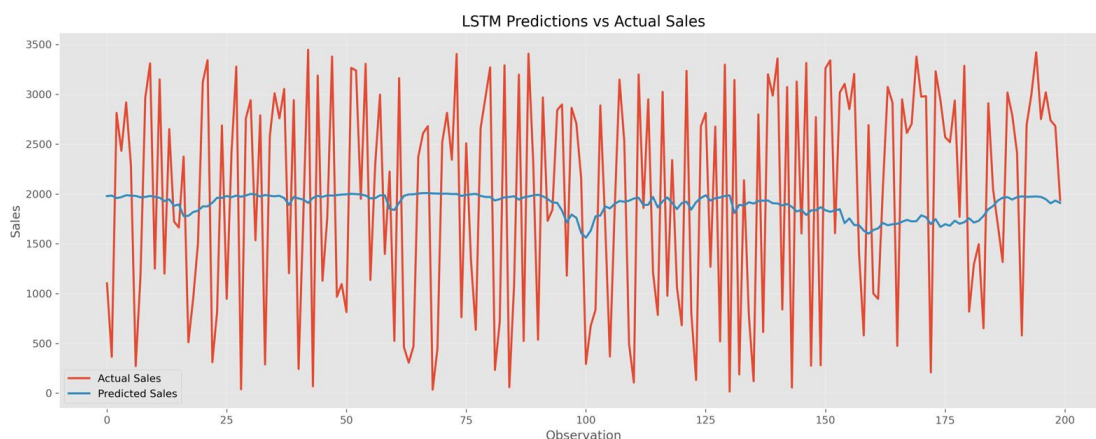


Figure 8.1 – LSTM Experiment 1 Predictions vs Actual Sales (Test Set)

8.3 Weaknesses

- The unscaled target combined with MSE loss produced very large loss values early in training, slowing effective optimization.

- Test R^2 of approximately 0.002 indicates the model explained almost none of the variance in sales — its predictions were only marginally better than always predicting the mean.
- The preprocessing pipeline (QuantileTransformer fitted for a tree-based model) was not specifically tuned for neural network training.
- The engineered lag and rolling features, while highly informative for LightGBM's split-based decision trees, do not automatically translate into equally useful signals for a recurrent architecture that already processes 30 days of raw sequential history — there may be redundancy or scale mismatch between these engineered features and the raw time steps.

These weaknesses directly motivated a second LSTM experiment (Chapter 9) with a preprocessing pipeline and architecture designed specifically for neural network training.

Chapter 9: LSTM Experiment 2

9.1 Improved Preprocessing

The second LSTM experiment replaced the QuantileTransformer used in Experiment 1 with a MinMaxScaler applied to the 19 input features, a scaling approach commonly preferred for neural networks because it bounds all inputs to a consistent [0, 1] range without altering the underlying feature distribution shape.

9.2 Target Scaling

Unlike Experiment 1, the target variable (Sales) was scaled independently using its own MinMaxScaler, fitted only on the training split. The network was trained to predict scaled sales values, with predictions inverse-transformed back to the original sales scale before evaluation. This is a standard and important practice for neural network regression, since unscaled targets with large magnitudes (up to ~3,500) can destabilize gradient-based optimization.

9.3 Improved Architecture

- LSTM layer 1: 128 units, return_sequences=True, input shape (30 × 19)
- Dropout: 30%
- LSTM layer 2: 64 units
- Dropout: 30%
- Dense hidden layer: 32 units, ReLU activation
- Output layer: 1 unit (linear)

This deeper, wider architecture (127,297 trainable parameters, versus 34,465 in Experiment 1) was trained with the Adam optimizer (initial learning rate 0.001) and MSE loss, using EarlyStopping (patience 10) and ReduceLROnPlateau (patience 5) callbacks for up to 100 epochs.

9.4 Results

Dataset	MAE	RMSE	R ²	SMAPE
Training	945.88	1069.59	0.0310	59.86%
Validation	945.58	1069.58	0.0180	60.11%
Testing	954.69	1082.29	0.0124	61.16%

The improvements in preprocessing, target scaling, and network capacity produced only a modest gain over Experiment 1: test MAE improved from 969.92 to 954.69 and test R² rose from 0.0017 to 0.0124 — still far below the LightGBM baseline's 0.8147. This result, explored further in Chapter 10, suggests the underperformance of the LSTM approach on this dataset is not primarily a matter of scaling or architecture depth, but reflects a more fundamental mismatch between the recurrent modelling paradigm and the structured, feature-engineered nature of this particular forecasting problem.

Chapter 10: Model Comparison

The table below summarizes the test-set performance of all three models, evaluated on an identical held-out test split using four standard regression metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), the coefficient of determination (R^2), and Symmetric Mean Absolute Percentage Error (SMAPE). SMAPE was preferred over standard MAPE because the dataset contains sales values very close to zero, which can cause MAPE to produce misleadingly large or unstable percentage errors.

Model	MAE	RMSE	R^2	SMAPE
LightGBM	353.58	478.57	0.8068	30.61%
LSTM Experiment 1	969.92	1088.14	0.0017	62.15%
LSTM Experiment 2	954.69	1082.29	0.0124	61.16%

Across every metric, LightGBM decisively outperformed both LSTM variants: its MAE was roughly 2.7 times lower than either LSTM model, its RMSE was roughly 2.3 times lower, and it explained over 80% of the variance in test-set sales compared with under 2% for the LSTM models. LSTM Experiment 2's improved preprocessing and deeper architecture produced a small improvement over Experiment 1 on every metric, but this improvement was marginal relative to the gap separating both LSTM models from LightGBM.

Chapter 11: Discussion

11.1 Why LightGBM Dominated

LightGBM's strong performance can be attributed to its ability to directly exploit the engineered lag and rolling-statistic features (`Sales_Lag_1/7/30`, `Rolling_Mean_7/30`, `Rolling_STD_7`) as independent, high-information splitting variables. Gradient-boosted decision trees excel at finding non-linear thresholds and interactions across a moderate number of well-engineered tabular features, and the feature importance analysis in Chapter 7 confirmed that both the raw weather variables and these engineered historical features contributed substantially to its predictions.

11.2 Why LSTM Struggled

Despite two rounds of experimentation — including changing the feature and target scaling strategy and substantially increasing model capacity — neither LSTM model came close to matching LightGBM's performance. Several factors likely contributed:

- Redundant sequence information: the LSTM already receives 30 days of raw sequential weather and sales history as its input; feeding it engineered lag/rolling features derived from that same history (as in this pipeline) creates redundancy that a recurrent network is not naturally positioned to exploit the way a tree-based splitting algorithm is.
- Sample size relative to model complexity: with roughly 17,000 training sequences, the dataset is comparatively small for a deep recurrent architecture, especially the 127,000-parameter Experiment 2 model, increasing the risk of the network converging to a broad, low-variance average prediction rather than learning fine-grained patterns.
- Noise-dominated target: daily sales exhibit very high observation-to-observation volatility (Chapter 3), and 30-day sequences of weather variables may carry a comparatively weak and diffuse signal relative to this noise, making gradient-based sequence learning harder to optimize than tree splits that can directly threshold on the most recent lag values.

11.3 Weather and Calendar Variables Are Tabular Data

The core insight from this comparison is that weather observations and calendar attributes, even organized into a 30-day sequence, remain fundamentally cross-sectional/tabular features rather than data with strong inherent sequential dependency of the kind LSTMs were designed for (e.g., natural language or long-range time series with strong autocorrelation structure). Once lag and rolling features explicitly encode the relevant recent history into a single row, a tabular model like LightGBM has direct, efficient access to exactly the information a recurrent network would otherwise need many training examples to learn implicitly through its hidden state.

11.4 Deep Learning Isn't Always the Best Choice

This project provides a concrete illustration of a well-established finding in applied machine learning: deep learning does not automatically outperform classical methods, particularly on small-to-moderate-sized structured/tabular datasets. Gradient-boosted trees remain a strong default choice for such problems, and the additional complexity, training time, and tuning burden of a recurrent neural network is only justified when the data and problem structure genuinely reward it — for example, very large datasets, or

sequences with rich, non-tabular structure such as raw text or continuous sensor streams. This project's results support using LightGBM as the production model for this business problem.

Chapter 12: Conclusion

12.1 Objectives Achieved

- Confirmed that weather variables, particularly humidity, are measurably associated with burger sales.
- Built a robust feature engineering pipeline incorporating calendar, lag, and rolling-statistic features.
- Developed and hyperparameter-tuned a LightGBM baseline achieving $R^2 \approx 0.81$ on held-out test data.
- Developed and iterated on two LSTM architectures, rigorously evaluating whether deep learning could improve on the tree-based baseline.
- Directly compared all three models using consistent, chronologically valid evaluation metrics (MAE, RMSE, R^2 , SMAPE).
- Prepared the best-performing model and its preprocessing artifacts for AWS SageMaker deployment.

12.2 Key Findings

- LightGBM substantially outperformed both LSTM models across every evaluation metric.
- Engineered lag and rolling-statistic features were among the most important predictors for the tree-based model.
- Humidity was the weather variable most strongly associated with sales; other weather variables contributed weaker but still non-trivial signal.
- Neither improved scaling nor increased LSTM model capacity closed the performance gap with LightGBM, pointing to a structural mismatch between the recurrent modelling approach and this tabular forecasting problem rather than a tuning deficiency.

12.3 Business Implications

The results support deploying the LightGBM model as the primary forecasting tool for operational decision-making. With an R^2 of roughly 0.81 and a SMAPE near 30% on unseen data, the model provides a meaningfully informative signal for next-day sales that can support inventory planning, staffing decisions, and supply-chain optimization across all ten regions, while the modest regional sales differences identified in Chapter 3 suggest that store-level or region-specific factors, beyond weather alone, may merit further investigation to close the remaining forecast error.

Chapter 13: Lesson Learnt

Throughout this project, several important technical and practical lessons were gained that extend beyond the development of predictive models. The project demonstrated that building a successful machine learning solution requires far more than selecting a sophisticated algorithm; careful data preparation, feature engineering, and rigorous evaluation were equally important in determining overall model performance.

One of the most significant lessons learned was the value of data preprocessing. Cleaning anomalous pressure values, removing duplicate records, engineering calendar-based variables, and creating lag and rolling statistical features had a substantial impact on the predictive performance of the models. This reinforced the understanding that the quality of the input data often has a greater influence on model accuracy than increasing model complexity.

The project also highlighted the importance of feature engineering for time-series forecasting. Historical sales information captured through lag features and rolling statistics proved to be highly informative, particularly for the LightGBM model. These engineered features enabled the model to capture temporal patterns that were not immediately apparent from the original weather variables alone.

Another important lesson was the need to prevent data leakage when working with time-series data. Rather than randomly splitting the dataset, a chronological train-validation-test split was used to ensure that future observations were never available during training. This approach produced a more realistic evaluation of model performance and better reflects real-world forecasting scenarios.

The comparison between LightGBM and the two LSTM experiments demonstrated that more complex models do not necessarily produce better results. Although LSTM networks are powerful sequence-learning models, they performed considerably worse than the gradient-boosted tree model on this structured tabular dataset. This experience emphasized the importance of selecting algorithms based on the characteristics of the data rather than assuming that deep learning will always outperform traditional machine learning methods.

Finally, this project provided valuable practical experience in building a complete machine learning pipeline, including exploratory data analysis, data cleaning, feature engineering, model development, hyperparameter tuning, evaluation using multiple performance metrics, model persistence, version control with GitHub, and deployment preparation using AWS SageMaker. Completing every stage of the workflow provided a deeper understanding of how machine learning projects are developed in professional environments and strengthened both technical and problem-solving skills.

References

- [1] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, Long Beach, CA, USA, 2017, pp. 3146–3154.
- [2] M. Abadi et al., “TensorFlow: A System for Large-Scale Machine Learning,” in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI 16)*, Savannah, GA, USA, 2016, pp. 265–283.
- [3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [4] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [5] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.
- [6] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.
- [7] Amazon Web Services, “Amazon SageMaker Developer Guide,” Amazon Web Services, Inc., 2024. [Online].